

Report conversation

In golang some types can be defined using `=` in the type definition. If I were to call these "aliased types", what you call types which reference another type but don't use `=`?

In Go (Golang), when you define a type using the `=` symbol, it's called an "alias declaration" or simply an "alias." This means the new type is essentially an alternate name for an existing type, and they are interchangeable.

For types that reference another type but don't use `=`, they are typically called "defined types" or "new types." In this case, the new type has the same underlying structure as the original type, but they are not interchangeable. This is used to create a distinct type with the same underlying data representation but with different methods or behaviors.

Here's a quick illustration:

1. Aliased Type:

go

 Copy code

```
type MyAlias = ExistingType
```

Here, `MyAlias` is just another name for `ExistingType`. They are the same type and can be used interchangeably.

2. Defined (or New) Type:

go

 Copy code

```
type MyDefinedType ExistingType
```

In this case, `MyDefinedType` is a new type. It has the same underlying data structure as `ExistingType`, but it is a distinct type. You cannot use one in place of the other without an explicit conversion.

Defined types are useful when you want to add specific methods to a type or when you want to make sure that your type is not accidentally used in place of another type, even if they have the same underlying structure.